# Building a RAG Chatbot: Voyage Travel Policy Assistant

## Welcome to the Voyage Team

Voyage is an innovative travel platform dedicated to moving beyond generic tourist advice. Our mission is to provide travelers with highly personalized, sensory-rich experiences that capture the true spirit of a destination.

With a growing customer base, and an increasing volume of support requests related to refund, cancellation and booking alteration questions, the team want to use AI to help answer these questions.

However, it is critical that the tool they build uses the Voyage travel policy document only to answer customer questions.

## Your Task

Your task is to build the **Travel Policy Assistant** for Voyage. You will create an AI assistant that allows customers to ask questions and get accurate, current answers without wading through pages of policies.

You must build a system that can:

- Use RAG to find the relevant sections, ensuring you pick up the most recent policies, not the outdated ones unless they are relevant.
- Run a set of test questions through the bot to prove that it answers correctly and complies with rules.

## Inspecting the data

We have `travel_policy.txt` in the workspace. Before chunking, inspect its structure. Understanding the document's formatting is the first step in choosing a good chunking strategy — a strategy that is wrong for this document's structure will produce chunks that mix sections, making it harder for the model to answer correctly.

Taking a look at the document, what formatting features could we use to split this text effectively?

## Strategic chunking

Since the document uses clear Markdown headers, we can use these to create meaningful chunks. This ensures that every piece of text stays attached to its section title.

## Setting up the Database

Now we need to convert these text chunks into searchable vectors. We will use Cosine Similarity to ensure our retrieval logic aligns with standard similarity scores (where 1.0 is a perfect match).

## Configuring the Retriever

To start with we are going to configure our baseline set-up. For this we are going to set the retriever to return only the most relevant result - the single most relevant section of the document.

## Designing the Prompt

We have our searchable database ready. Now we need to give the AI instructions on how to use it.

We will start with a standard prompt that instructs the model to answer user questions using the provided context.

## Verification: The Golden Test Set

Ad-hoc testing shows roughly whether the assistant works. A golden test set lets you measure pass rate consistently across changes. This is the foundation of LLMOps: you cannot improve what you cannot measure.

We grade two types of question:

- **Correctness:** Does the answer match the reference answer?
- **Refusal:** Did the assistant correctly decline an out-of-scope question?

| index | Question | Result | Reason |
|---|---|---|---|
| 0 | Can I get a refund for a Saver ticket? | PASS | To assess whether the submission meets th |
| 1 | What is the refund policy for a Standard ticket cancelled 24 hours before departure? | FAIL | To assess whether the submission meets th |
| 2 | How many checked bags are included for Business class? | PASS | To assess whether the submission meets th |
| 3 | What is the policy on quantum teleportation? | PASS | To assess whether the submission meets th |
| 4 | What is the best pizza in Rome? | PASS | To assess the submission based on the pro |

Rows: 5 Expand

### Tuning: Does K Matter?

We built the retriever with k=1. But we saw that some of the tests didn't pass. There are a number of things we can adjust in a RAG pipeline. How does adjusting the number of retrieved responses impact the restults?

Let's try k=3 and re-run the same evaluation.

| index | Question |
|---|---|
| 0 | Can I get a refund for a Saver ticket? |
| 1 | What is the refund policy for a Standard ticket cancelled 24 hours before departure? |
| 2 | How many checked bags are included for Business class? |
| 3 | What is the policy on quantum teleportation? |
| 4 | What is the best pizza in Rome? |

Rows: 5 Expand

### Experiment Tracking

In production you would use MLflow, Weights & Biases, or LangSmith for this. The concept is the same regardless of tooling: record every experiment with enough metadata to reproduce it, and compare results in a structured way.

Here we create a comparison table. The table is what matters: it shows at a glance which configuration performed better, and gives you an audit trail if you need to justify a choice later.

| i... | timestamp | model | chunking | experiment_name |
|---|---|---|---|---|
| 0 | 2026-05-06 18:24:46 | gpt-4o-mini | MarkdownHeaderTextSplitter | baseline |
| 1 | 2026-05-06 18:24:49 | gpt-4o-mini | MarkdownHeaderTextSplitter | baseline |

Rows: 2 Expand

### Discussion

- Which configuration would you choose for production, and why?
- Where does the pipeline still fail? What would you change in the prompt or retrieval strategy?
- What additional metrics would you track if this assistant were handling real employee queries?